

HunyuanImage-3.0 AR->DiT KV-cache Reuse: Performance Breakdown Report

PR #2949 | vllm-project/vllm-omni | 2026-04-23

1. Experimental setup

Item	Value
Hardware	4 x NVIDIA L20X (143 GB), driver 570.133.20
Software	vllm 0.19.0, vllm-omni PR #2949, torch 2.10.0+cu128, NCCL 2.27.3
Stage topology	AR TP=2 on GPUs 0-1, DiT TP=2 on GPUs 2-3
Model	tencent/HunyuanImage-3.0-Instruct (MoE, 64 experts, EP=2)
Modality	img2img (IT2I)
Input image	735 x 1104 RGB, official assets/demo_instruct_imgs/input_0_0.png
Prompt	"■■■■■■■■..." (the T2I prompt from run_demo_instruct.sh)
Prompt length	140 chars / 101 tokens (user content only)
AR prefill length	~6690 tokens (system prompt + image tokens + user + assistant prefix)
Sampling	temperature 0.6, top-p 0.95, top-k 1024, bot_task=think_recaption
Diffusion	50 steps, guidance 5.0, output auto-snapped to 1216 x 832
KV transport	SharedMemoryConnector (single-node)

2. Input characterisation

Input	Value
Image pixels	735 x 1104 = 811,440 px (RGB, ~2.4 MB raw)
Text prompt chars	140 (Chinese + ASCII mixed)
Text prompt tokens	101 (HunyuanImage-3 tokenizer, BPE-like)
AR-side joint sequence tokens (prefill)	~6690 (sys ~6312 + image ~275 + user 101 + "Assistant: <think>" ~3)
AR-generated tokens (CoT)	424-481 (varies with sampling / CFG companion batching)
DiT image tokens	3953 (= timestep + patches for 1216 x 832 at patch stride 16)
DiT total joint sequence	~10700-10800 tokens

3. End-to-end timing breakdown (single request)

All values are wall-clock seconds measured from log timestamps. The **engine init** column is the one-off cost to load model weights + compile MoE, incurred once per server start -- it is not on the per-request critical path in production. The **E2E (excl. init)** column is the per-request latency that matters for QPS / latency SLOs.

Run	Path	Sampling	AR tokens	Engine init	AR wall	ar2diffusion bridge	kv ecv	DiT preproc+denoise	E2E (excl. init)
A	no-KV	temp=0.6	424	85.6 s	34.0 s	1803 ms	-	45 ms + ~57.3 s	91.0 s
D	KV-reuse	temp=0.6	481	89.5 s	39.0 s	1851 ms	0.37 s	44 ms + ~26.5 s	65.0 s
E	no-KV	greedy	481	86.3 s	38.0 s	1901 ms	-	48 ms + ~57.3 s	95.0 s
F	KV-reuse	greedy	443	86.1 s	37.0 s	1707 ms	0.37 s	44 ms + ~26.0 s	63.0 s

Notes on the numbers:

- **AR wall** is from `_handle_add_request` at the orchestrator down to the `[ar2diffusion] Request 0: ... log line`, i.e. prefill + CoT decode. In KV-reuse mode the AR also streams the KV cache out concurrently, which is why D/F are slightly slower than A/E at comparable token counts.

- **ar2diffusion bridge** is the stage-input-processor callback that serialises `ar_generated_text`, multi-modal data, and sampling params for the DiT. It also includes the AutoTokenizer fallback decode when `detokenize: false` on the AR stage (~1.7-1.9 s, dominated by tokenizer construction + Chinese BPE decode; tokenizer is cached from 2nd request on).
- **KV recv** on the DiT side is overlapped with ar2diffusion via the orchestrator's async event loop, so only the tail (~0.37 s for the 445 MB primary, 0.29 s for the 407 MB CFG companion, max across TP ranks) shows up as extra time.
- **DiT preproc** is the schedule/attention-mask build plus the 3 DiT special-token pad in KV-reuse. **DiT denoise** covers 50 denoising steps of joint attention over text + image tokens.

4. KV transport detail (KV-reuse only)

For every KV-reuse request the AR stage streams two tensors to the DiT:

- **Primary KV** (positive branch) -- the AR prefill of the actual prompt: ~445 MB (443-481 AR tokens produce a slightly bigger cache because of the CoT; the text prefill is what dominates).
- **CFG companion KV** (uncond / negative branch) -- the AR prefill of the <cfg>-substituted companion prompt: ~407 MB.

SharedMemoryConnector (measured): 4xL20X single-node, AR TP=2 -> DiT TP=2 (4 ranks total), two concurrent transfers per rank.

Transfer	Size / rank	Wall time (max rank)	Throughput / rank (avg)	Observed latency on DiT side (wait + link)
Primary KV	445 MB	0.45 s	~966 MB/s	0.37 s wait (link 0.37 s)
CFG companion KV	407 MB	0.41 s	~972 MB/s	0.29 s wait (link 0.29 s)
Total per request	~852 MB	~0.45 s (overlapped)	~970 MB/s per rank	~0.37 s (parallel, overlapped with ar2diffusion)

Mooncake / RDMA connector: the code path in `mooncake_transfer_engine_connector.py` is wired up by the PR but was **not functionally benchmarked** on this 4x L20X single-node machine (no RDMA fabric available). For a 400-Gbps RDMA fabric an order-of-magnitude estimate based on the 852 MB total payload would be ~20 ms end-to-end for the primary+companion pair at line rate, i.e. ~20x faster than the current SharedMemoryConnector figure. This is theoretical; real-world benchmarking is flagged in the PR validation scope as future work.

5. Side-by-side timing comparison

"Non-reuse baseline" = config A (no-KV, sampling). "KV-reuse" = config D (KV-reuse, sampling). Same prompt, same seed, same target resolution.

Phase	Non-reuse (A)	KV-reuse (D)	Delta
Engine init (one-off)	85.6 s	89.5 s	+3.9 s (more processes + CFG companion init)
AR generate + prefill	34.0 s	39.0 s	+5.0 s (AR also streams KV out; CFG companion also prefill)
ar2diffusion bridge	1.80 s	1.85 s	+0.05 s (tokenizer fallback + CFG KV reshape)
KV recv (DiT side)	0 s	0.37 s	+0.37 s
DiT preprocess	0.045 s	0.044 s	equal
DiT denoise (50 steps)	~57.3 s	~26.5 s	-30.8 s (2.16x faster)
E2E per request (excl. init)	91.0 s	65.0 s	-26.0 s (1.40x faster)

Key insight: KV-reuse pays a one-time tax of ~5.4 s on the AR side (extra AR decoding + KV send) plus ~0.4 s on KV receive, and in return saves ~30.8 s on DiT denoise. The net win on a single IT2I request is **26 s (1.40x)**. At 50 steps the DiT savings scale roughly linearly with the step count, so for more aggressive 30-step variants the absolute gain shrinks but the relative gain grows as the AR overhead is amortised against proportionally less DiT time.

6. Projection to other workloads

The measurements above are single-request on an IT2I workload; the following rules-of-thumb apply for re-scoping:

- **Larger image:** DiT denoise time scales ~linearly in image tokens (image tokens grow as $H*W / \text{patch_size}^2$). AR time does not move. KV payload is invariant to output image size (it's the text KV). So at 1536x1536 (~3x more image tokens) the KV-reuse speed-up in absolute seconds grows further.
- **Longer prompt:** AR prefill grows ~linearly in prompt tokens; KV payload grows proportionally. Text KV at 20k tokens is ~1.3 GB/branch; over SHM that is still ~1.4 s, negligible vs a 50-step denoise. Over RDMA it is ~30 ms.
- **Multi-request batching:** the DiT benefit is approximately per-request (each request saves its own first-step text forward); the AR tax also scales per-request. The KV-reuse ratio therefore remains ~2x at the DiT level regardless of batch size.

7. Reproducibility

All numbers above come from the four raw log files on the **pr-2949-breakdown-assets** branch of the fork. The configs used:

- hunyuan_image3_it2i.yaml (no-KV 4-GPU, devices=0..7 in repo; the TP=2 variant used here is in /root/local_test_configs/hunyuan_image3_it2i_tp2.yaml)
- hunyuan_image3_it2i_kv_reuse.yaml (KV-reuse 4-GPU, devices=0..7 in repo; TP=2 variant is hunyuan_image3_it2i_kv_reuse_tp2.yaml)

Command template:

```
python examples/offline_inference/hunyuan_image3/end2end.py --modality img2img
--image-path input_0_0.png --prompts "..." --stage-configs-path <config.yaml> --steps
50 --guidance-scale 5.0 --seed 42 --bot-task it2i_think --sys-type en_unified
```

Appendix A: per-run raw timing

Run	AR init_time	AR tokens	AR gen_chars	2nd diff wall (ms)	DiT preproc (ms)	DiT denoise (ms)	DiT total (ms)
A (no-KV, sampling)	85.6 s	424	749	1803	45.3	57276	57322
D (KV-reuse, sampling)	89.5 s	481	819	1851	44.3	26495	26540
E (no-KV, greedy)	86.3 s	481	840	1901	48.1	57340	57389
F (KV-reuse, greedy)	86.1 s	443	786	1707	43.7	26041	26085

Appendix B: KV transfer log lines (raw)

D (sampling): primary 445 MB @ 966 MB/s avg, companion 407 MB @ 972 MB/s avg.

F (greedy): primary 443 MB @ 966 MB/s avg, companion 407 MB @ 973 MB/s avg.

(Throughput numbers are the per-rank averages across the 4 TP ranks (2 send + 2 rcv).)

Appendix C: Sample images

2x2 grid of A (no-KV, sampling seed 42), B (no-KV, sampling seed 42, run 2), C (no-KV, sampling seed 123), D (KV-reuse, sampling seed 42).



A: no-KVseed=42 run1



B: no-KVseed=42 run2



C: no-KVseed=123



D: KV-reuse seed=42